
PROGRAMACIÓN

Para estudiantes de ingeniería

- 1ª Edición -

Guillermo Manns



Contenido

Prefacio	5
Introducción	6
1 Linux	7
1.1 Directorios Linux	8
1.1.1 Directorio raíz	8
1.1.2 Binarios y comandos esenciales	8
1.1.3 Directorios personales de los usuarios.	8
1.1.4 Directorio boot	9
1.1.5 Directorio dev	9
1.1.6 Directorio etc	9
1.1.7 Directorio lib	9
1.1.8 Directorio media	9
1.1.9 Directorio mount	9
1.1.10 Directorio opt	9
1.1.11 Directorio root	10
1.1.12 Directorio sbin	10
1.1.13 Directorio de archivos temporales	10
1.1.14 Directorio user	10
1.1.15 Directorio var	10
1.2 Navegación básica	11
1.2.1 ¿Dónde estoy? ¿Qué hago aquí?	11
1.2.2 Crear carpetas y archivos	11
1.2.3 Manipular archivos y directorios	12
1.3 Edición de texto	13
1.3.1 Editor nano	13
1.3.2 Permisos de ejecución	13
1.3.3 Procesos	13
1.3.4 Tuberías	13
1.4 Gestión de usuarios, permisos y redes	14
1.5 Gestión de almacenamiento	15
1.6 Gestión de procesos y servicios	16

1.7	Redirección y automatización básica	17
2	Bash	18
2.1	Fundamentos del lenguaje Bash	19
2.1.1	El “Shebang” y la salida estándar	19
2.1.2	Variables y la regla del “espacio prohibido”	19
2.1.3	Comillas simples vs. dobles y expansión	20
2.1.4	Captura de entrada del usuario (read)	21
2.1.5	Argumentos posicionales (\$1, \$2, \$@)	22
2.2	Operaciones y Control de Flujo	23
2.2.1	Operaciones aritméticas básicas	23
3	Instalación de Arch Linux	24
3.1	Conectar a Wi-Fi y Verificar Entorno	25
3.2	Particionado y Formateo del Disco	26
3.3	Instalación del Sistema Base	27
3.4	Configuración del Sistema Interno (chroot)	27
3.5	Configuración del Sistema	28
3.6	Configuración de Usuarios y Privilegios	29
3.7	Interfaz Gráfica, Optimización de CPU y Entornos	29
3.8	Alternativa Futura (Gestor de Ventanas Qtile)	30

Versión 1.2.3 - 27 de julio de 2026

Prefacio

Apuntes de programación

Introducción

Capítulo 1

Linux

1.1 Directorios Linux

```
/
├── bin
├── boot
├── dev
├── etc
├── home
├── lib
├── media
├── mnt
├── opt
├── sbin
├── tmp
├── usr
└── var
```

1.1.1 Directorio raíz

```
/
```

Es el punto de inicio del sistema de archivos de Linux. Aquí se encuentran todos los demás directorios del sistema.

1.1.2 Binarios y comandos esenciales

```
/bin
```

Aquí se encuentran los archivos ejecutables (binarios) que son esenciales para el funcionamiento del sistema. Estos archivos incluyen comandos básicos utilizados por los usuarios y programas del sistema.

1.1.3 Directorios personales de los usuarios.

```
/home
```

Cada usuario tiene un subdirectorio aquí (por ejemplo, /home/user).

1.1.4 Directorio boot

```
/boot
```

Contiene los ejecutables del Kernel y todos los archivos necesarios para ejecutar el sistema.

1.1.5 Directorio dev

```
/dev
```

contiene archivos especiales que brindan acceso a dispositivos de hardware.

1.1.6 Directorio etc

```
/etc
```

Contiene los archivos de configuración del sistema, también puede almacenar archivos de configuración de aplicaciones instaladas.

1.1.7 Directorio lib

```
/lib
```

Contiene todas las librerías compartidas, éstas son necesarias para la ejecución de binarios.

1.1.8 Directorio media

```
/media
```

En este directorio se montan los dispositivos de media extraíbles (como USB).

1.1.9 Directorio mount

```
/mnt
```

Sirve para montar sistemas de archivos temporalmente. Está pensado para que el administrador del sistema monte dispositivos de forma transitoria.

1.1.10 Directorio opt

```
/opt
```

Aquí se almacenan archivos de aplicaciones externas (de terceros). Los programas instalados en /opt suelen tener su propia estructura de directorios interna.

1.1.11 Directorio root

```
/root
```

Este es el directorio personal del super usuario (administrador del sistema).

1.1.12 Directorio sbin

```
/sbin
```

Este directorio contiene los binarios del sistema. Cumple la misma función que /bin, pero en este caso para el super usuario.

1.1.13 Directorio de archivos temporales

```
/tmp
```

Aquí es donde el sistema operativo y otros programas almacenan sus archivos temporales. Al reiniciar el sistema se vacía por completo.

1.1.14 Directorio user

```
/usr
```

Este directorio contiene librerías, ejecutables, recursos compartidos, pero que no son esenciales para el arranque del sistema, pero que sí son necesarios para su funcionamiento correcto.

1.1.15 Directorio var

```
/var
```

Este directorio contiene archivos variables, es decir, que se van modificando continuamente. Por ejemplo: archivos .log (eventos del sistema).

1.2 Navegación básica

1.2.1 ¿Dónde estoy? ¿Qué hago aquí?

Comando	Descripción
<code>pwd</code>	(print working directory) Muestra la ruta del directorio actual.
<code>ls</code>	(list) Lista los archivos y directorios en el directorio actual.
<code>ls -l</code>	Lista los archivos y directorios con detalles adicionales, como permisos, propietario, tamaño y fecha de modificación.
<code>cd</code>	(change directory) Cambia el directorio actual. Por ejemplo, <code>cd /home/usuario</code> te lleva al directorio <code>/home/usuario</code> .
<code>cd ..</code>	Sube un nivel en la jerarquía de directorios.
<code>cd ~</code>	Te lleva al directorio home del usuario actual.

1.2.2 Crear carpetas y archivos

Comando	Descripción
<code>mkdir nombre_carpeta</code>	Crea un nuevo directorio con el nombre especificado.
<code>touch nombre_archivo</code>	Crea un nuevo archivo vacío con el nombre especificado.
<code>rmdir nombre_carpeta</code>	Elimina un directorio vacío.
<code>find ruta -name nombre_archivo</code>	Busca un archivo o directorio con el nombre especificado en la ruta dada.

1.2.3 Manipular archivos y directorios

Comando	Descripción
<code>></code>	Redirige la salida de un comando a un archivo, sobrescribiendo su contenido. Por ejemplo, <code>echo "Hola" > archivo.txt</code> escribirá “Hola” en <code>archivo.txt</code> , reemplazando cualquier contenido previo.
<code>>></code>	Redirige la salida de un comando a un archivo, agregando al final de su contenido. Por ejemplo, <code>echo "Mundo" >> archivo.txt</code> agregará “Mundo” al final de <code>archivo.txt</code> .
<code>cat > nombre_archivo</code>	Crea un nuevo archivo y permite escribir contenido en él. Para finalizar la escritura, presiona Ctrl+D .
<code>echo</code>	Repite en pantalla lo que se escriba. Por ejemplo: <code>echo 'Hola mundo' > codigo.txt</code>
<code>mv origen destino</code>	Mueve un archivo o directorio desde la ubicación de origen a la ubicación de destino. También se puede usar para renombrar archivos o directorios.
<code>cp origen destino</code>	Copia un archivo o directorio desde la ubicación de origen a la ubicación de destino.
<code>rm nombre_archivo</code>	Elimina el archivo especificado.
<code>rm -r nombre_carpeta</code>	Elimina el directorio especificado y todo su contenido de manera recursiva.
<code>clear</code>	Limpia la pantalla de la terminal, eliminando todo el contenido visible.
<code>less nombre_archivo</code>	Muestra el contenido de un archivo de texto de manera paginada, permitiendo desplazarse hacia arriba y hacia abajo.

Observación 1.1:

En Arch Linux requiere instalar el paquete `less` con `sudo pacman -S less`.

1.3 Edición de texto

1.3.1 Editor nano

Comando	Descripción
nano	Editor de texto en consola. Se debe instalar mediante: <code>sudo pacman -S nano</code>

Ejemplo 1.1: script básico

```
#!/bin/bash
echo ''Hola usuario''
echo ''Tu sistema lleva encendido:''
uptime
```

1.3.2 Permisos de ejecución

1.3.3 Procesos

1.3.4 Tuberías

1.4 Gestión de usuarios, permisos y redes

1.5 Gestión de almacenamiento

1.6 Gestión de procesos y servicios

1.7 Redirección y automatización básica

Capítulo 2

Bash

2.1 Fundamentos del lenguaje Bash

2.1.1 El “Shebang” y la salida estándar

Todo script de Bash debe comenzar con una línea especial llamada Shebang (`#!/`). Esta línea le indica al sistema qué intérprete debe usar para ejecutar el código que viene a continuación. Para Bash, la ruta absoluta estándar es `/bin/bash`.

Ejemplo 2.1: script básico

```
#!/bin/bash
# Esto es un comentario en Bash

echo ''Comenzando el curso de Bash''
```

Los scripts deben ser guardados en formato `.sh`

2.1.2 Variables y la regla del “espacio prohibido”

En Bash, puedes crear variables para almacenar datos, pero existe una regla de oro que confunde a muchos programadores que vienen de otros lenguajes: no debe haber absolutamente ningún espacio alrededor del signo igual (`=`) en la asignación.

Si pones un espacio, Bash pensará que el nombre de tu variable es un comando que intentas ejecutar y fallará. Por defecto, todas las variables en Bash se tratan como cadenas de texto (strings). Para acceder al valor que guardaste en la variable, debes anteponer el símbolo de dólar (`$`).

Ejemplo 2.2: script básico

```
#!/bin/bash

# Correcto (sin espacios)
lenguaje='Bash'
version=5

# Llamar a la variable usando $
echo ''Estamos aprendiendo $lenguaje''
```

2.1.3 Comillas simples vs. dobles y expansión

En la clase anterior viste cómo usar el símbolo `$` para obtener el valor de una variable dentro de la salida estándar. Sin embargo, Bash interpreta este símbolo de manera diferente dependiendo del tipo de comillas que lo rodeen.

Comillas dobles (`"..."`): Son comillas “débiles”. Permiten lo que se conoce como expansión de variables. Si escribes `$variable` dentro de comillas dobles, Bash lo leerá, lo reconocerá y lo reemplazará por su valor real.

Comillas simples (`'...'`): Son comillas “fuertes” o estrictamente literales. Todo lo que esté dentro de ellas se interpreta como texto puro. Si escribes `$variable` dentro de comillas simples, Bash ignorará el símbolo de dólar e imprimirá literalmente el texto `"$variable"`.

Ejemplo 2.3: script básico

```
#!/bin/bash

# Definimos una variable
sistema="Arch Linux"

# Expansion de la variable
echo 'Estoy ejecutando codigo en $sistema'

# Texto literal sin expansion
echo 'Estoy ejecutando codigo en $sistema'
```

2.1.4 Captura de entrada del usuario (read)

Para que un script de Bash reciba información directamente del usuario que lo está ejecutando, usamos el comando incorporado `read`. Cuando Bash encuentra este comando, pausa la ejecución del script y espera a que el usuario escriba algo en la terminal y presione Enter.

Lo que el usuario escribe se guarda automáticamente en la variable que indiques después del comando `read`.

Un truco muy útil: en lugar de usar un `echo` para hacer la pregunta y luego un `read` en la siguiente línea, `read` tiene una opción (un flag) `-p` (de prompt) que te permite imprimir el mensaje y capturar la variable en la misma línea.

Ejemplo 2.4: script básico

```
#!/bin/bash

# Metodo tradicional
echo "Escribe una palabra:"
read palabra1
echo "Escribiste: $palabra1"

# Metodo mas limpio con -p
read -p "Escribe otra palabra: " palabra2
echo "Tambien escribiste: $palabra2"
```

2.1.5 Argumentos posicionales (\$1, \$2, \$@)

Hasta ahora, le has pedido datos al usuario con `read` mientras el script está en ejecución. Pero en Bash, es muy común pasarle la información al script en el mismo momento en que lo ejecutas desde la terminal. Estos se llaman argumentos posicionales.

Bash asigna variables automáticas a las palabras que escribes después del nombre del script:

`$0`: Contiene el nombre del script mismo.

`$1`, `$2`, `$3`, ... : Contienen el primer, segundo, tercer argumento, respectivamente.

`$@`: Representa todos los argumentos que se le pasaron.

`$#`: Contiene el número total de argumentos recibidos.

Ejemplo 2.5: script básico

```
#!/bin/bash

# Se ejecuta asi en la terminal: ./script.sh Arch Linux
echo "El script en ejecucion es: $0"
echo "El primer argumento es: $1"
echo "El segundo argumento es: $2"
echo "Me has pasado un total de $# argumentos."
echo "Todos los argumentos juntos son: $@"
```

2.2 Operaciones y Control de Flujo

2.2.1 Operaciones aritméticas básicas

En Bash, todas las variables son tratadas como texto (strings) por defecto. Si intentas hacer `resultado="5 + 5"` y lo imprimes, Bash literalmente mostrará `5 + 5`.

Para que el intérprete evalúe una expresión matemática (suma, resta, multiplicación, división entera), debes usar la Expansión Aritmética. Esto se logra encerrando la operación entre dobles paréntesis precedidos por un signo de dólar: `$(( expresion ))`.

Un detalle muy cómodo: dentro de los dobles paréntesis, no necesitas usar el símbolo `$` para llamar a las variables. Bash ya sabe que estás en un contexto matemático.

(Nota: Bash nativo solo trabaja con números enteros. Para cálculos con decimales o límites matemáticos más complejos, se suele llamar a herramientas externas como `bc`, `awk` o usar Python, pero para la lógica del script, los enteros suelen bastar).

Ejemplo 2.6: script básico

```
#!/bin/bash

x=10
y=3

# Suma
suma=$(( x + y ))

# Multiplicacion
multiplicacion=$(( x * y ))

# Division entera (trunca los decimales)
division=$(( x / y ))

# Modulo (Resto de la division)
modulo=$(( x % y ))

echo "La suma es: $suma"
echo "La division entera es: $division"
```

Capítulo 3

Instalación de Arch Linux

3.1 Conectar a Wi-Fi y Verificar Entorno

El primer paso requiere establecer una conexión activa a internet mediante el entorno interactivo de instalación y comprobar la arquitectura de la placa madre.

- **Verificación de Arquitectura EFI:** para corroborar que el sistema inició en modo UEFI, listamos las variables de firmware correspondientes.

```
ls /sys/firmware/efi/efivars
```

- **Conexión a la Red Inalámbrica:** utilizamos la herramienta interactiva `iwctl` para autenticar la tarjeta de red y conectarnos al punto de acceso:

```
iwctl
# Dentro del entorno iwctl:
station wlan0 scan
station wlan0 get-networks
station wlan0 connect NombreRed
quit
```

Observación 3.1: Instalación de Arch en Lenovo Ideapad 100

- **Solución de Errores Específicos de Hardware (Módulos Realtek)** En laptops Lenovo IdeaPad 100 o equipos con tarjetas inalámbricas Realtek conflictivas (como la `rtl8723be`), es necesario forzar la antena correcta para evitar pérdidas de señal:

```
modprobe -r rtl8723be
modprobe rtl8723be ant_sel=2
ip link set wlan0 up
```

- **Verificación de Conectividad** Comprobamos el envío y recepción de paquetes ICMP para asegurar el enlace:

```
ping -c 3 google.com
```

3.2 Particionado y Formateo del Disco

Trabajaremos sobre el esquema del disco físico principal indexado como `/dev/sda`.

- **Limpieza de la Tabla de Particiones** Eliminamos cualquier estructura previa de almacenamiento y creamos una nueva tabla de particiones de tipo **GPT** (GUID Partition Table):

```
parted /dev/sda
# Dentro del entorno parted:
mklabel gpt
quit
```

- **Estructura de Particiones en cfdisk** Accedemos a la herramienta visual `cfdisk` seleccionando la tabla de particiones de tipo **GPT**:

```
cfdisk /dev/sda
```

Se define la siguiente distribución del almacenamiento:

- **/dev/sda1:** Nueva → 1 GB → Type: *EFI System*.
 - **/dev/sda2:** Nueva → 4 GB → Type: *Linux swap*.
 - **/dev/sda3:** Nueva → Resto del disco → Type: *Linux filesystem*.
- **Formateo de Particiones** Asignamos los sistemas de archivos correspondientes a cada bloque estructurado:

```
# Particion EFI (FAT32)
mkfs.fat -F 32 /dev/sda1

# Particion Swap (Intercambio)
mkswap /dev/sda2
swapon /dev/sda2

# Particion Raiz (ext4)
mkfs.ext4 /dev/sda3
```

Observación 3.2:

Aplicar cambios escribiendo `write`, confirmar con `yes` y presionar `Quit` para salir.

3.3 Instalación del Sistema Base

Preparamos los puntos de montaje e inyectamos el núcleo del sistema operativo.

- **Montaje Inicial de Unidades** Montamos la raíz e inicializamos el directorio interno para la partición de arranque:

```
mount /dev/sda3 /mnt
mkdir -p /mnt/boot
mount /dev/sda1 /mnt/boot
```

- **Instalación mediante pacstrap** Descargamos los paquetes esenciales en la raíz montada:

```
pacstrap -K /mnt base linux linux-firmware nano
```

- **Generación del Archivo de Puntos de Montaje (fstab)** Registramos los identificadores únicos de las particiones para los arranques subsecuentes:

```
genfstab -U /mnt >> /mnt/etc/fstab
```

3.4 Configuración del Sistema Interno (chroot)

- **Acceso al Entorno Virtualizado** Ingresamos formalmente al disco duro del equipo para estructurar las directivas de usuario y localización.

```
arch-chroot /mnt
```

- **Zona Horaria y Reloj del Sistema** Establecemos el huso horario correspondiente a Chile continental y sincronizamos los relojes de hardware:

```
ln -sf /usr/share/zoneinfo/America/Santiago /etc/localtime
hwclock --systohc
```

- **Localización e Idioma** Editamos las plantillas generadoras de idioma del sistema:

```
nano /etc/locale.gen
```

- Descomentar desmarcando el caracter '#' en la línea:

```
es_CL.UTF-8 UTF-8
```

- **Compilación de Idiomas** Compilamos los idiomas habilitados y fijamos la variable de entorno global:

```
locale-gen
echo "LANG=es_CL.UTF-8" > /etc/locale.conf
```

3.5 Configuración del Sistema

- **Configuración del Teclado en Consola (TTY)** Para que la distribución del teclado en modo texto reconozca la disposición en español:

```
echo "KEYMAP=es" > /etc/vconsole.conf
```

- **Identidad de Red (Hostname)** Asignamos un nombre único a la máquina en la red local y asignamos la contraseña maestra de `root`:

```
echo "Arch-Lenovo" > /etc/hostname  
passwd
```

- **Gestión de Redes Definitiva** Instalamos los demonios correspondientes para asegurar la conectividad posterior:

```
pacman -S networkmanager iw wpa_supplicant  
systemctl enable NetworkManager
```

- **Instalación y Despliegue del Cargador de Arranque (GRUB)** Instalamos el gestor binario de arranque y escribimos sus bloques en la partición EFI activa:

```
pacman -S grub efibootmgr  
grub-install --target=x86_64-efi --efi-directory=/boot --bootloader-id=GRUB  
grub-mkconfig -o /boot/grub/grub.cfg
```

- **Conclusión de la Fase Base y Reinicio** Salimos del aislamiento del entorno virtual, desmontamos de forma recursiva y reiniciamos el hardware:

```
exit  
umount -R /mnt  
reboot
```

Observación 3.3:

Desconectar la unidad de almacenamiento USB (pendrive) en cuanto la pantalla cambie a negro.

3.6 Configuración de Usuarios y Privilegios

Tras el inicio autónomo del disco interno, accedemos con las credenciales de **root**.

- **Conexión Inicial Mediante NetworkManager** Invocamos la interfaz semi-gráfica para reactivar el enlace Wi-Fi sin dependencias de terceros:

```
nmtui
# Seleccionar "Activar una conexion" -> Elegir Red
                                -> Ingresar Password
                                -> Salir.
```

- **Creación del Usuario Personal y Permisos sudo** Instalamos la utilidad de elevación de privilegios y creamos el espacio de trabajo secundario:

```
pacman -S sudo
useradd -m -G wheel -s /bin/bash guillermo
passwd guillermo
```

- Para otorgar capacidades de administración al grupo del usuario, ejecutamos el editor seguro:

```
EDITOR=nano visudo
# Buscar y descomentar removiendo el '#' de la línea:
# %wheel ALL=(ALL:ALL) ALL
```

Salimos de la sesión mediante **exit** e iniciamos sesión con el usuario definitivo (**guillermo**).

3.7 Interfaz Gráfica, Optimización de CPU y Entornos

- **Servidor Gráfico y Entorno de Escritorio Ligero (XFCE)** Instalamos el servidor de despliegue Xorg junto al entorno tradicional optimizado y el gestor de accesos interactivo:

```
sudo pacman -S xorg-server xfce4 xfce4-goodies lightdm lightdm-gtk-greeter
sudo systemctl enable lightdm
```

- **Optimización Avanzada de Energía para Procesadores Intel Celeron** Para regular la frecuencia de los núcleos de forma dinámica, mitigar temperaturas elevadas y prolongar los ciclos de descarga en discos mecánicos (HDD):

```
sudo pacman -S tlp tlp-rdw
sudo systemctl enable tlp.service
```

- **Configuraciones del Entorno de Ventanas de Escritorio** Para forzar la correcta traducción de caracteres y mapear las señales del teclado en el ecosistema X11 al español latinoamericano:

```
localectl set-x11-keymap latam
```

- **Despliegue del Navegador Web Principal** Inyección del navegador nativo de código abierto:

```
sudo pacman -S firefox
```

3.8 Alternativa Futura (Gestor de Ventanas Qtile)

Para migrar a una maquetación en mosaico minimalista basada íntegramente en scripts dinámicos en lenguaje Python:

- **Instalación de Componentes de Qtile y Terminal Acelerada**

```
sudo pacman -S qtile python-psutil python-xdg python-iwlib alacritty
```

- **Generación del Espacio de Trabajo en Python** Estructuramos la jerarquía del archivo `config.py` heredando la configuración por defecto proporcionada por el sistema:

```
mkdir -p ~/.config/qtile  
cp /usr/share/doc/qtile/default_config.py ~/.config/qtile/config.py
```

Observación 3.4:

Tras cerrar sesión en LightDM, conmutar la sesión activa a “Qtile” e ingresar con las credenciales locales.